
System Requirements Specification

for

LaunchName: Idea-to-Domain-to-Agent Launch Platform

Version 1.0

Prepared by:

Arya Wadhwani
Saloni Belliappa Bolakaranda

Revision History

Version	Date	Comment
0.1	08/29/2026	Initial prototype
0.2	09/02/2026	name.com Core API and Claude integration
0.3	09/03/2026	Agent identity and discovery layer
0.4	10/05/2026	Navigation, accessibility and visual revision
1.0	10/06/2026	Release version

Table of Contents

Revision History.....i

Table of Contents.....ii

List of Figuresiii

List of Tablesiv

1. Introduction1

 1.1 Project Scope1

 1.2 Acronyms and Definitions.....3

2. Overall Description5

 2.1 System Context.....5

 2.2 System Functions6

 2.3 Operating Environment7

 2.4 Design and Implementation Constraints8

 2.5 Assumptions and Dependencies.....9

3. External Interface Requirements.....9

 3.1 Data Elements for External Interface11

4. Internal System Features.....13

 4.1 Launch Wizard.....13

 4.2 Domain Service16

 4.3 Registration Orchestrator18

 4.4 DNS Manager21

 4.5 Agent Registry23

 4.6 Agent Gateway25

5. Nonfunctional Requirements27

 5.1 Performance Requirements28

 5.2 Security Requirements29

Appendix A. User Interface Screens31

Appendix B. Requirements Traceability Matrix37

Appendix C. Verification Summary.....39

List of Figures

Figure 1-1: The fragmented launch journey before LaunchName	1
Figure 1-2: Project scope and interactions between LaunchName and other stakeholders	2
Figure 1-3: Example data that is provided by LaunchName	3
Figure 2-1: LaunchName system context diagram.....	6
Figure 2-2: Major system functions by area	7
Figure 2-3: Operating environment tiers	8
Figure 2-4: Registration-period rules by extension and API limits.....	9
Figure 2-5: Assumptions and dependencies.....	9
Figure 3-1: LaunchName high-level response structure	11
Figure 3-2: Agent discovery record medium-level structure	12
Figure 3-3: Supported user flows	13
Figure 4-1: Launch wizard steps, back navigation and lock point.....	14
Figure 4-2: Agent identity step with sectioned layout and help tips	15
Figure 4-3: Domain discovery sequence.....	17
Figure 4-4: Domain discovery with equal-size cards, scores and filters.....	17
Figure 4-5: Registration and launch sequence.....	19
Figure 4-6: Launch Center after a successful registration.....	20
Figure 4-7: DNS record change sequence with server-side validation	21
Figure 4-8: DNS manager, Advanced view	22
Figure 4-9: Agent publish and verify sequence	24
Figure 4-10: Agent profile showing the four verification checks	24
Figure 4-11: Agent-to-agent message sequence	26
Figure 4-12: Agent Connect after a completed exchange.....	27
Figure 5-1: Measured response times per endpoint	28
Figure 5-2: Security trust boundary.....	29
Figure A-1: Landing page with milestone tracker and animated idea field	31
Figure A-2: Scroll-driven 3D DNS lookup on the landing page.....	32
Figure A-3: Brand discovery step	32
Figure A-4: Confirm step with registration period, WHOIS privacy and help tips.....	33
Figure A-5: Registration progress driven by real API results	33

LaunchName

System Requirements Specification for LaunchName

Figure A-6: Operator dashboard overview	34
Figure A-7: DNS manager, Simple view, including the agent discovery record	34
Figure A-8: Add DNS record dialog	35
Figure A-9: Agent directory	35
Figure A-10: Landing page on a 390-pixel-wide phone	36
Figure C-1: Latest test run	39

List of Tables

Table 1-1: Acronym list	3
Table 1-2: Glossary of terms	4
Table 3-1: External interface requirements	10
Table 3-2: High-level structure of an outgoing API response.....	11
Table 3-3: Supported user flow reference table	12
Table 4-1: Launch Wizard functional requirements.....	15
Table 4-2: Domain Service functional requirements	18
Table 4-3: Registration Orchestrator functional requirements.....	20
Table 4-4: DNS Manager functional requirements	22
Table 4-5: Agent Registry functional requirements	25
Table 4-6: Agent Gateway functional recommendations	27
Table 5-1: Performance requirements.....	28
Table 5-2: Security requirements.....	29
Table B-1: Requirements traceability matrix	37
Table C-1: Automated test suites.....	39

1. Introduction

LaunchName is a web platform that turns a one-sentence product idea into a registered domain with configured DNS and, optionally, an AI agent identity that other agents can discover and talk to. It was built for a hackathon centered on the name.com API, and every domain, price and DNS record it shows comes from a live call to name.com's Core API rather than a static list or a mockup.

This System Requirements Specification (SyRS) describes the requirements the platform implements. It was derived from the working implementation, its design notes, and the verification carried out during development: an automated suite of 71 unit and integration tests, and end-to-end browser runs against the name.com sandbox. Requirements are written so that each one can be traced to the code that implements it and the test that verifies it (Appendix B).

1.1 Project Scope

Launching a new product online today takes a founder through a chain of disconnected tools: a naming tool to find a brand, a registrar to buy a domain, a DNS console to hand-edit records, a host to point a site at, and, increasingly, an agent registry so AI agents can find the product. Each step lives in a separate account, and values are copied by hand from one to the next. This is represented in the “before” diagram shown in Figure 1-1.

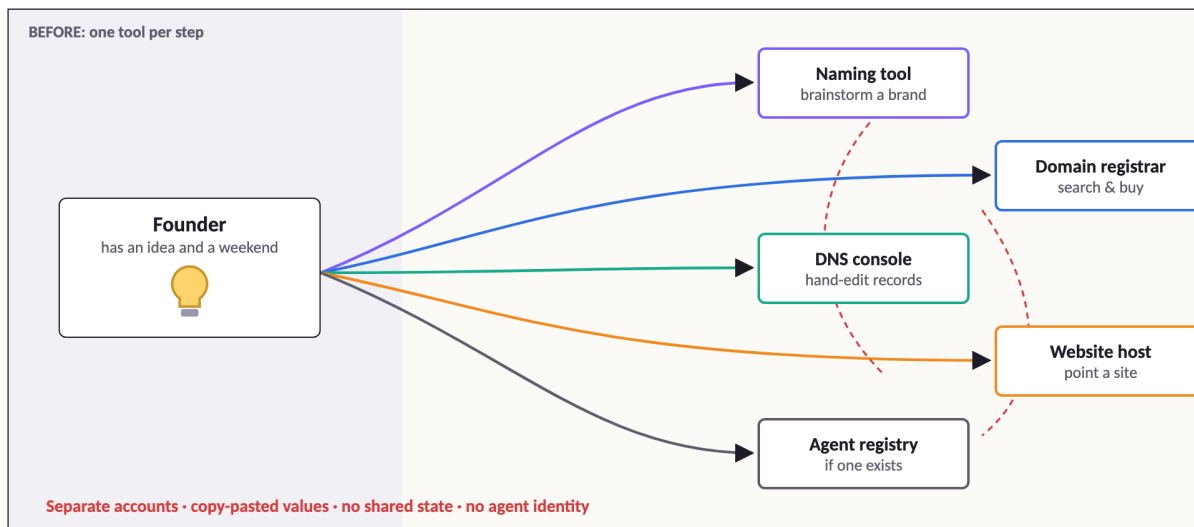


Figure 1-1: The fragmented launch journey before LaunchName

LaunchName replaces this chain with one guided flow. Standardizing the journey keeps every step consistent and lets each step build on the last: the brand chosen in step one becomes the search term for domains, the domain becomes the address of the agent, and the agent's DNS record becomes its proof of identity. The platform's scope covers the interactions between its users, the name.com Core API, Anthropic's Claude API and other AI agents, as shown in Figure 1-2.

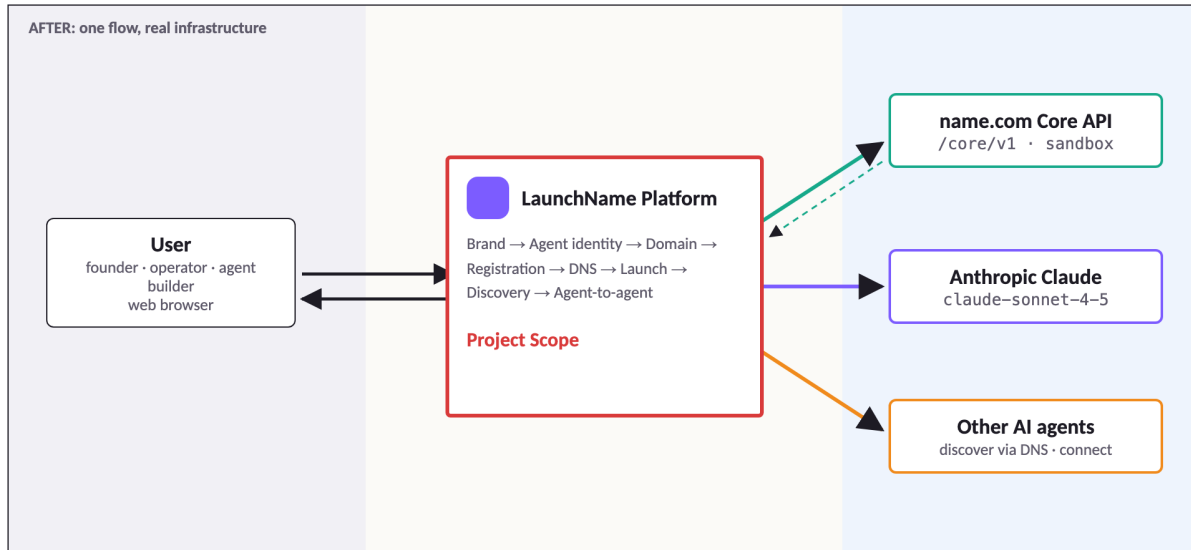


Figure 1-2: Project scope and interactions between LaunchName and other stakeholders

The data the platform produces and exchanges falls into three groups, depicted in Figure 1-3: infrastructure data from name.com (availability, pricing, registration, DNS records and per-extension rules), agent discovery data (agent manifests, discovery TXT records, verification state and handshake logs), and AI-generated data from Claude (brand names, taglines, agent capabilities and in-character replies). Payment processing, hosting of the user's production website, and the operation of name.com's or Anthropic's own services are considered out of scope.

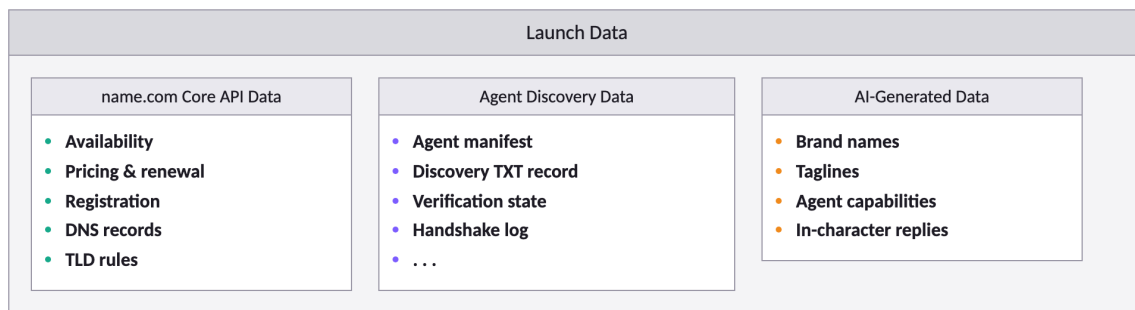


Figure 1-3: Example data that is provided by LaunchName

1.2 Acronyms and Definitions

Table 1-1 provides a list of all of the acronyms used in this document. Table 1-2 defines many of the terms that are used in this document where the authors identified some potential ambiguities or confusion.

Table 1-1: Acronym list

Acronym	Definition
AI	Artificial Intelligence
ANS	Agent Name Service
API	Application Programming Interface
CNAME	Canonical Name (DNS alias record)
DNS	Domain Name System
DNS-AID	DNS-based agent discovery proposal described by Seethiraju et al. (Verisign, arXiv:2606.02314)
E2E	End-to-End
HCI	Human–Computer Interaction
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
LLM	Large Language Model
MCP	Model Context Protocol
MUI	Material UI (React component library)
REST	Representational State Transfer
SyRS	System Requirements Specification
TLD	Top-Level Domain (the extension, e.g. .com, .ai)
TLS	Transport Layer Security
TTL	Time To Live (DNS caching period, seconds)
TXT	Text record (DNS record type holding free-form text)
UI	User Interface
UX	User Experience
WCAG	Web Content Accessibility Guidelines
WHOIS	Not an acronym; the public lookup service listing who registered a domain

Table 1-2: Glossary of terms

Term	Definition
Function	A goal or objective accomplished by an implementation of a product or service.
Feature	A system or subsystem that is utilized in order to achieve a function.
Founder	A user who starts from an idea and goes through the launch wizard.
Operator	A user who manages registered domains and DNS records from the dashboard.
Launch	The record of one idea taken through registration: brand, domain, DNS and (optionally) agent.
Wizard step	One of six stages of the launch flow: Brand, Agent, Domain, Confirm, Register, Launch.
Furthest step	The latest editable step a user has reached; saved progress can be resumed up to it.
Agent identity	The name, description, capabilities, protocols and endpoint declared for a domain's AI agent.
Publication point	Where an agent lives under a domain: the apex (“@”) or a subdomain such as “travel”.
Discovery marker	The DNS TXT record at _agent-discovery that points agents at a domain's manifest.
Live / Demo agent	A live agent answers at its own reachable endpoint; a demo agent is answered in character by Claude through LaunchName's gateway and is always labeled as such.
Live / Mock mode	Live mode calls name.com; mock mode uses an in-repository provider that mirrors the Core API when no credentials are configured.
Requirement	A necessary condition that the system needs to meet to operate as designed.
Stakeholder	An organization or person with an interest in utilizing the platform.

2. Overall Description

Standardization of the platform's external interfaces (to name.com, to Anthropic and to the user's browser) is the focus of this document. Internal data communication between subsystems is described to provide context. All calls to external services are made from server-side code; the browser communicates only with the platform's own API routes.

2.1 System Context

A system context diagram is shown in Figure 2-1. In this diagram, the external systems are communicating with the platform through defined interfaces. The internal subsystems are

communicating data through the blue arrows, whereas a dashed blue arrow indicates an optional data flow. Some subsystems contain components, which are depicted in gray. This context diagram guides the design of the following sections.

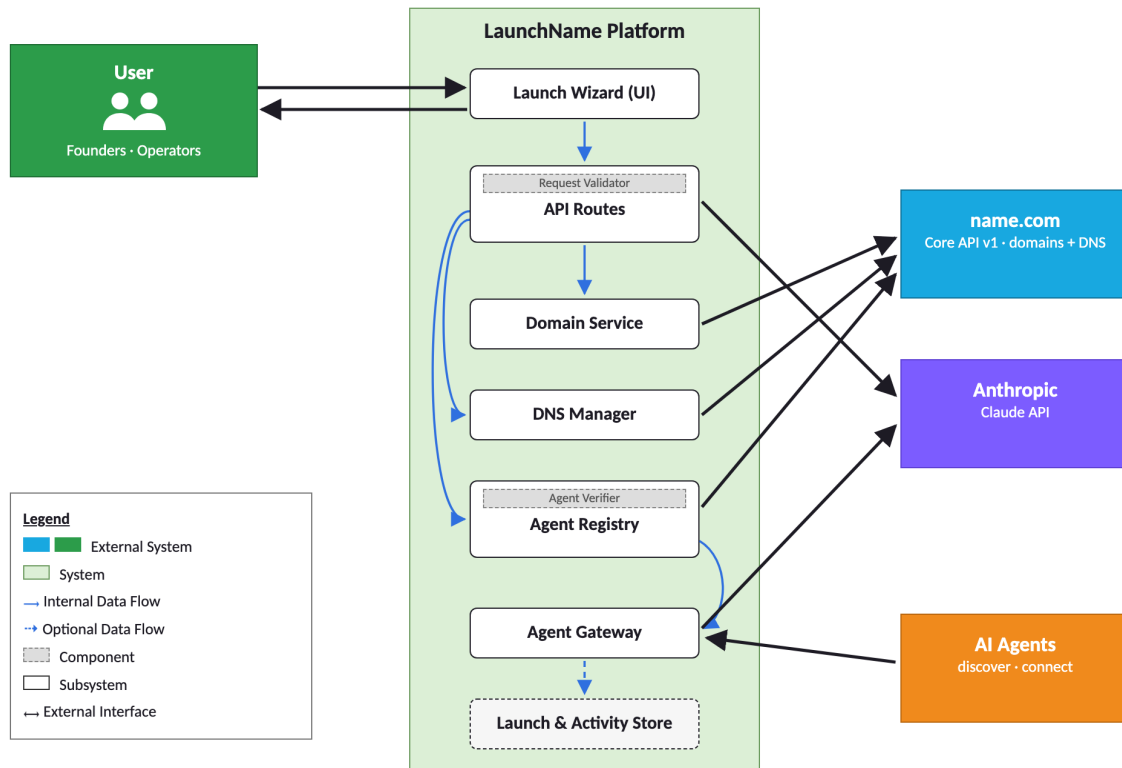


Figure 2-1: LaunchName system context diagram

2.2 System Functions

LaunchName supports the following major system functions, grouped in Figure 2-2:

- Generate brandable names and taglines from a one-sentence idea.
- Draft an agent profile (description, category and capabilities) from the same idea.
- Check live domain availability and pricing for up to 24 candidate domains per brand.
- Enforce each extension's allowed registration period, read live from name.com.
- Register a domain with optional WHOIS privacy, protected by an idempotency key.
- Configure DNS automatically after registration (A, CNAME and ownership TXT records).
- Create, read, update and delete DNS records with server-side validation.
- Publish agent discovery metadata as a DNS TXT record and verify ownership by reading it back.
- Let one agent discover and message another through a clearly labeled live or demo connector.
- Save wizard progress so users can go back and resume without losing their edits.

- Optionally fall back to a mock provider and an offline name generator when credentials are absent.
- Report API activity and capability coverage to operators.

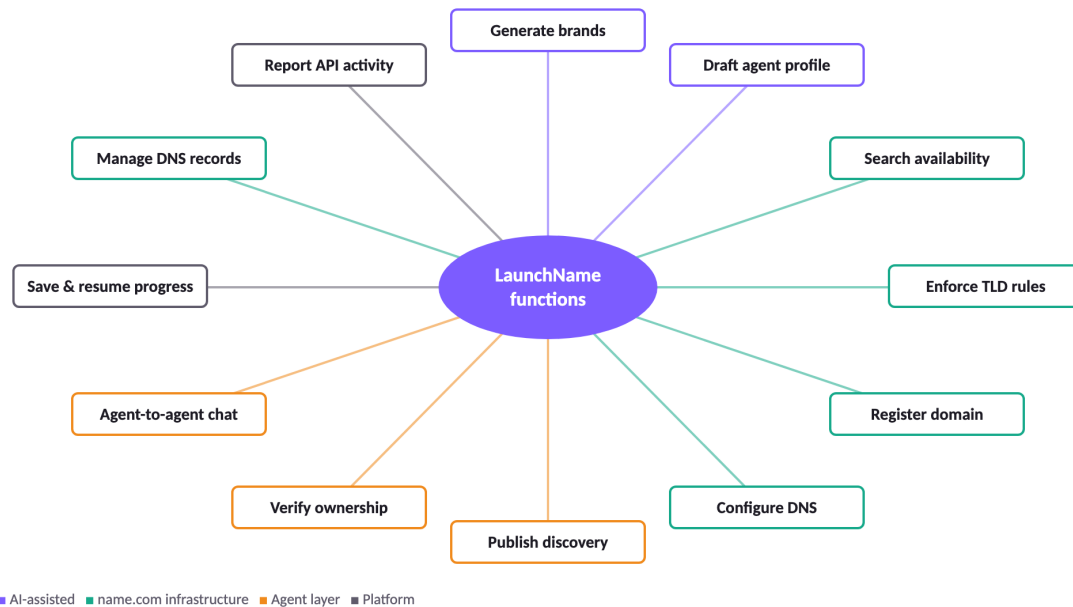


Figure 2-2: Major system functions by area

2.3 Operating Environment

The platform runs as a Next.js 14 application on Node.js 20 and can be deployed to Vercel's serverless runtime or as a Docker container built from node:20-alpine. Users need nothing beyond a modern web browser on desktop or mobile. Software subsystems coexist inside one application; the platform defines and makes available its interfaces in the form of REST endpoints that accept and return JSON. Figure 2-3 shows the three tiers.

LaunchName Project (devpost26)

Final System Requirements Specification for LaunchName

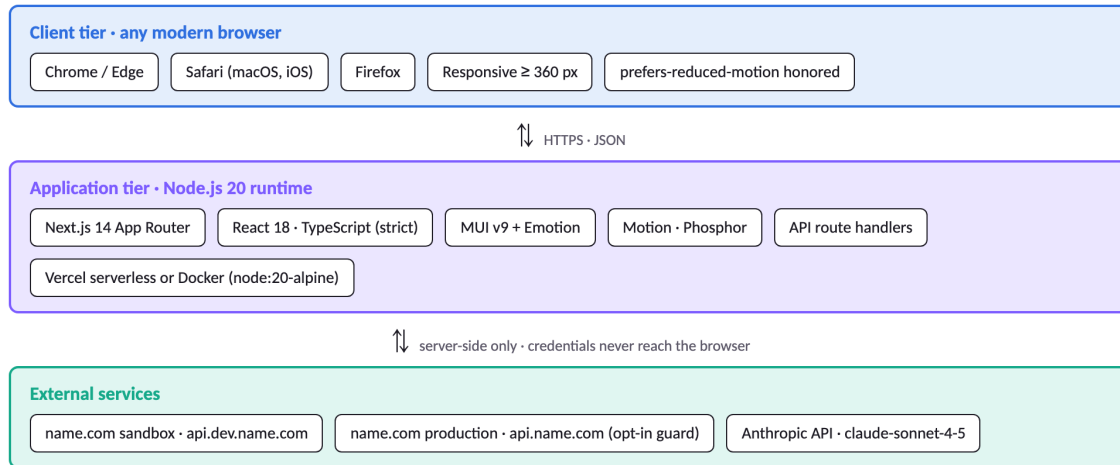


Figure 2-3: Operating environment tiers

2.4 Design and Implementation Constraints

Implementers should be aware of the following constraints: name.com rate limits, per-extension registration rules, sandbox DNS behavior, AI response latency, in-memory state and accessibility. name.com documents limits of 20 requests per second and 3,000 per hour; the platform retries rate-limited and server-error responses up to three times and aborts any request after 12 seconds. Registration periods differ by extension (Figure 2-4): .ai requires at least two years and .co allows at most five, so the allowed values must be read from name.com rather than assumed.

Registrations made in name.com's sandbox never propagate to public DNS, so agent endpoints on sandbox domains are not publicly reachable; the platform must say so rather than report them as live. Claude takes several seconds to generate brands (median 8.7 s), so every AI step needs a contextual loading state. Launch and agent records are held in server memory, which is sufficient for a single long-running process but can be lost between cold starts on serverless hosting. Finally, all text must meet WCAG AA contrast and all motion must stop when the user prefers reduced motion.

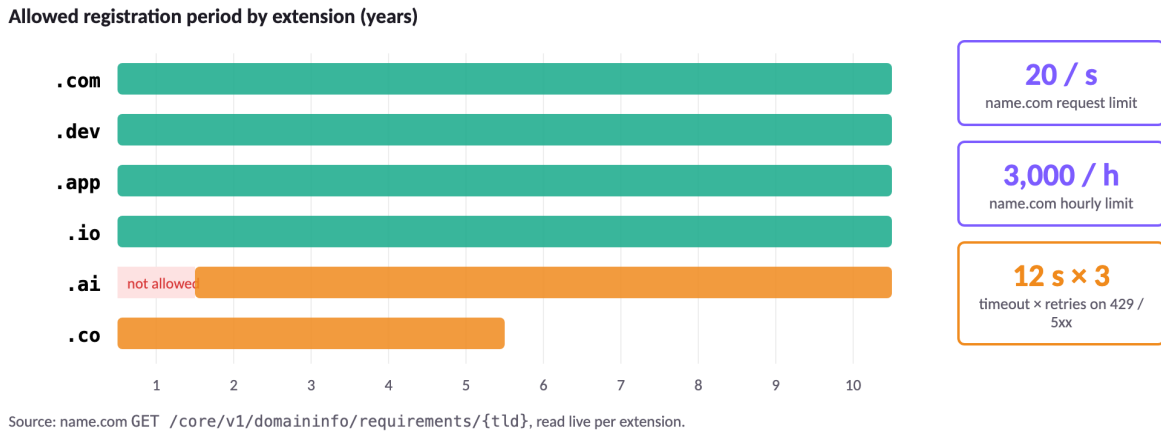


Figure 2-4: Registration-period rules by extension and API limits

2.5 Assumptions and Dependencies

It is assumed that name.com's Core API and Anthropic's API are reachable over the internet, that valid sandbox credentials and an Anthropic API key are configured on the server, and that users run a browser with JavaScript and session storage enabled. When credentials are missing, the platform degrades instead of failing: a deterministic mock provider stands in for name.com and an offline generator stands in for Claude, and the interface labels which mode is active. Figure 2-5 summarizes these dependencies.

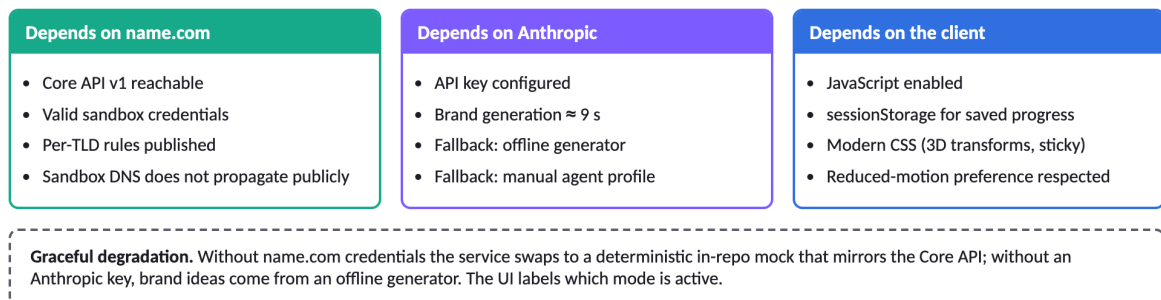


Figure 2-5: Assumptions and dependencies

3. External Interface Requirements

The external interfaces connect the platform to the user's browser, to name.com and to Anthropic. Each external interface shall have the set of requirements listed in Table 3-1. References to requirements throughout this document are structured according to the following guidelines:

- <REQ> or <OPT>: REQ for requirements, OPT for optional recommendations.
- <Sublabel>: Short, three letter label for the high-level category of the entry.
- <Major number>: The major number is incremented for every major capability captured by a requirement.

- <Minor number>: The minor number starts with 1 for the first entry of a major capability and is incremented by every entry that supports it.
- <Optional sub-number>: Sub-numbers are created when an entry supports a minor capability.

Table 3-1: External interface requirements

ReqID	Requirement
REQ-EXT-1.1	The platform shall communicate with name.com exclusively through the Core API v1 (/core/v1/*) and shall never call the deprecated v4 API.
REQ-EXT-1.2	The platform shall authenticate to name.com using HTTP Basic Authentication built from server-side credentials.
REQ-EXT-1.3	The platform shall default to name.com's sandbox host and shall refuse production calls unless they are explicitly enabled (ALLOW_NAMECOM_PRODUCTION=true).
REQ-EXT-1.4	The platform shall abort any name.com request that takes longer than 12 seconds.
REQ-EXT-1.5	The platform shall retry name.com requests that return 429 or 5xx up to 3 times, honoring the Retry-After header.
REQ-EXT-1.6	The platform shall send an X-Idempotency-Key with every domain purchase so a retried request cannot purchase twice.
REQ-EXT-1.7	The platform shall read each extension's allowed registration periods and privacy support from GET /core/v1/domaininfo/requirements/{tld}.
REQ-EXT-2.1	The platform shall call Anthropic's Messages API (model claude-sonnet-4-5) only from server-side code.
REQ-EXT-2.2	The platform shall remain usable when the Claude API is unavailable, using an offline brand generator and allowing agent profiles to be written by hand.
REQ-EXT-3.1	The platform shall expose its functions to the browser as REST endpoints that accept and return JSON.
REQ-EXT-3.2	The platform shall return errors as { error: { code, message } } with an HTTP status mapped from the error code.
REQ-EXT-3.3	The platform shall never return credentials, raw upstream error bodies or stack traces to the browser.

3.1 Data Elements for External Interface

Every response from the platform's API contains either a payload or an error, never both. Table 3-2 captures the structure of a response sent by the platform to the browser.

Table 3-2: High-level structure of an outgoing API response

Message Portion	Data Element
Success	Payload: results, record, agent, launch, manifest or requirements object
Error	code (VALIDATION, CONFLICT, RATE_LIMITED, TIMEOUT, ...)
	message (plain-language, safe to show to users)
Transport	HTTP status mapped from the code (e.g. 400, 409, 429, 502, 504)

A representation of the high-level response structure is shown in Figure 3-1.

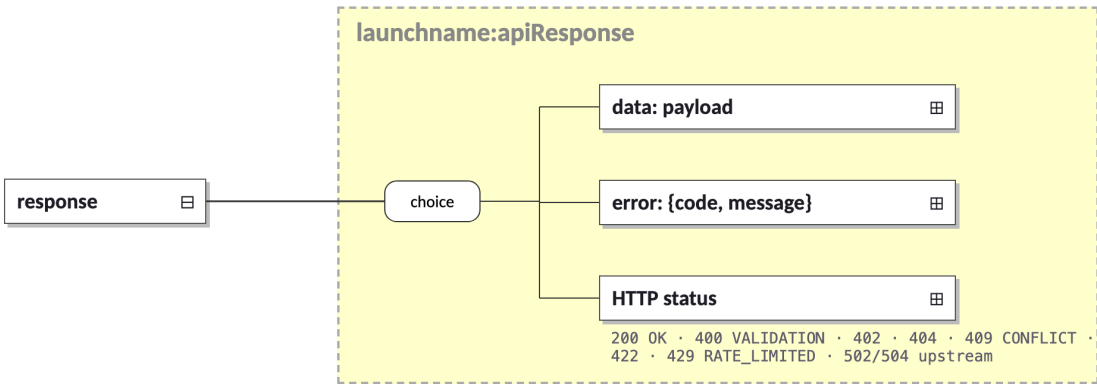


Figure 3-1: LaunchName high-level response structure

The platform's main custom data structure is the agent discovery record. Each of its components has the subcomponents represented in Figure 3-2, including the compact text form that is published to DNS.

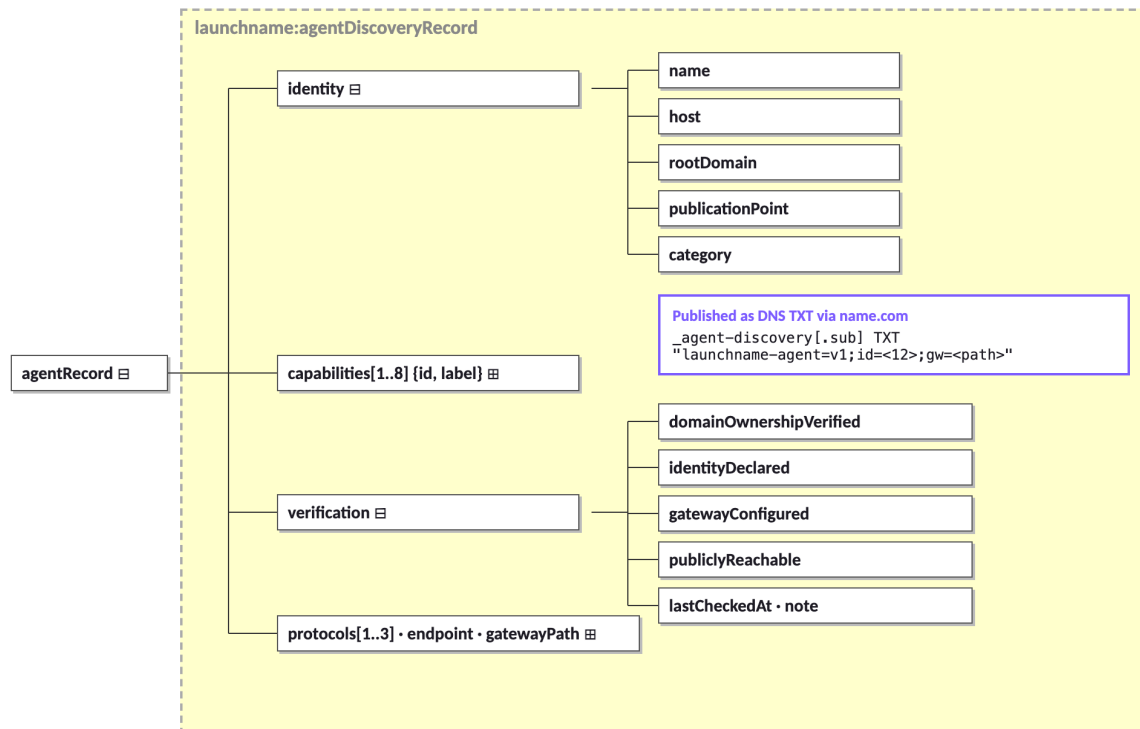


Figure 3-2: Agent discovery record medium-level structure

3.1.1 Supported User Flows

To support the flows identified as valuable during design, specific endpoints and payloads are defined in this document. The flows in Table 3-3 rely on the interfaces above; Figure 3-3 shows how users move through them.

Table 3-3: Supported user flow reference table

Flow Reference Name	Flow Name	Description
BRAND	Brand Discovery	Turns a one-sentence idea into brandable names, taglines and personality traits generated by Claude.
AGENT	Agent Identity	Lets a founder declare an AI agent for the domain: name, description, capabilities and communication methods.
DSRCH	Domain Discovery	Checks live availability and pricing for every candidate domain and scores each for people and for agents.
REG	Registration and Launch	Purchases the domain, configures DNS, publishes and verifies the agent, and opens the Launch Center.
DNSM	DNS Management	Lets operators view, add, edit and delete records in plain-language or raw form.
DISC	Agent Discovery	Lets anyone find agents by what they can do and inspect their verification state.
A2A	Agent-to-Agent Connect	Runs a visible handshake and exchanges a message with another agent.

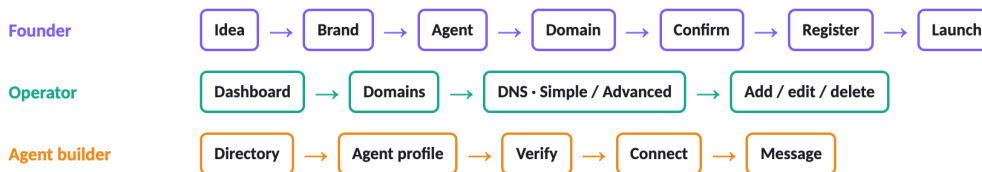


Figure 3-3: Supported user flows

4. Internal System Features

In this section, each internal subsystem and its components are outlined, including a brief description, the priority of implementation, the sequence of events following a user interaction, and the associated functional requirements. The subsystem names are summarized below along with an indication of whether or not the modules are optional:

- Launch Wizard [Required]
 - Saved Progress (Required)
- Domain Service [Required]
 - Mock Provider (Optional)
- Registration Orchestrator [Required]
- DNS Manager [Required]
 - Record Validator (Required)

- Agent Registry [Required]
 - Agent Verifier (Required)
- Agent Gateway (Optional)

4.1 Launch Wizard

The launch wizard guides a founder from an idea to a live domain in six steps. It holds all choices in one shared state so that each step builds on the last, saves that state after every change, and lets the user move back and forward without losing work. Every option it presents carries a help tip that explains what the option means.

4.1.1 Priority

The launch wizard is of high priority because it is the primary way users reach every other subsystem.

4.1.2 Stimulus/Response Sequences

When a user submits an idea, the wizard starts a new launch and requests brand names. Each later step loads its data once for the current brand; returning to a step shows the saved data instead of requesting it again. When the user picks a different brand, data that depended on the old brand is discarded. Once registration starts, the earlier steps lock because the domain has been purchased. Figure 4-1 shows these transitions and Figure 4-2 shows the Agent step.

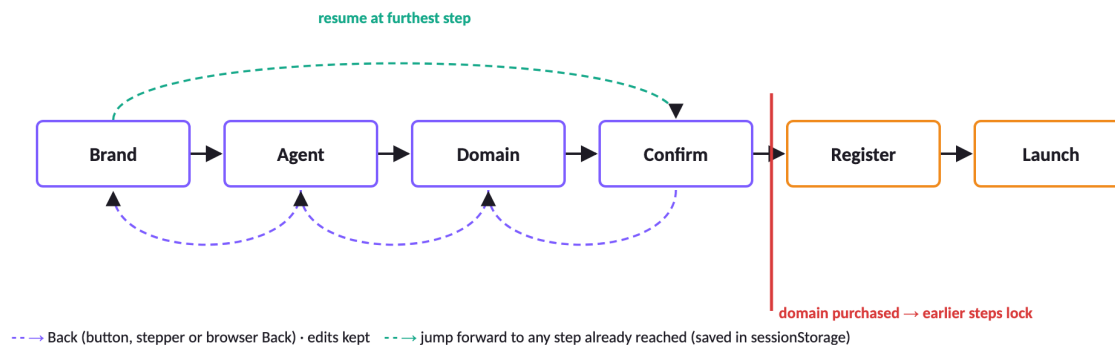


Figure 4-1: Launch wizard steps, back navigation and lock point

LaunchName Project (devpost26)

Final System Requirements Specification for LaunchName

LaunchName Brand Agent 3 Domain 4 Confirm 5 Register 6 Launch Powered by name.com

← Back to Brand

STEP 2 OF 6 - AGENT IDENTITY

What should your agent be able to do?

A domain isn't just where your site lives anymore. It can also be the address where other AI agents find and talk to yours.

Also create an AI agent identity for this domain ☒ ⓘ
Optional. Turn it off to launch just a website.

Identity ⓘ
How your agent introduces itself.

Agent name
Roost

Description
Roost helps renters manage their lease obligations by tracking important deadlines, sending timely reminders, and organizing rental documents.

Suggested by Claude from your idea. Edit freely; your changes are saved.

Capabilities 4 ⓘ
What other agents can ask yours to do. Add at least one.

Lease Deadline Tracking ⓘ Renewal Reminders ⓘ Document Organization ⓘ Payment Scheduling ⓘ

Add a capability (e.g. Itinerary Generation) Add

Figure 4-2: Agent identity step with sectioned layout and help tips

4.1.3 Functional Requirements

The functional requirements for the launch wizard are shown in Table 4-1.

Table 4-1: Launch Wizard functional requirements

Requirement ID	Description
REQ-IWZ-1.1	The wizard shall guide users through six steps: Brand, Agent, Domain, Confirm, Register and Launch.
REQ-IWZ-1.2	The wizard shall save progress to session storage after every change and restore it after a page reload.
REQ-IWZ-1.3	The wizard shall let users return to any earlier editable step through a Back button, the progress stepper or the browser's Back button.
REQ-IWZ-1.4	The wizard shall keep all user edits when navigating back and forward and shall not request data again that is already loaded for the same idea or brand.
REQ-IWZ-1.5	The wizard shall discard agent and domain data when the user selects a different brand.
REQ-IWZ-1.6	The wizard shall ignore late API responses for an idea or brand the user has moved away from.
REQ-IWZ-1.7	The wizard shall lock earlier steps once registration starts, and the browser's Back button shall leave the wizard rather than reopen locked steps.
REQ-IWZ-1.8	A reload during registration shall not submit the purchase again.
REQ-IWZ-1.9	The wizard shall explain every option with a help tip that opens on hover, keyboard focus and touch.
OPT-IWZ-1.10	The wizard may cycle typed example ideas in the idea field until the user focuses it.

4.2 Domain Service

The domain service turns a brand into a set of candidate domains, checks each one with name.com, and ranks the results for two audiences: people, who need a memorable and trustworthy address, and AI agents, which benefit from AI-native extensions and clean names. It is the only subsystem that talks to name.com's search and pricing endpoints.

4.2.1 Priority

The domain service is of high priority because registration, DNS and agent identity all depend on the domain it helps the user choose.

4.2.2 Stimulus/Response Sequences

When the user continues to the Domain step, the service builds the candidate list, checks availability in one batch, scores each available domain and marks a best pick for people and a best pick for agents. The user can then filter by extension group and by availability, compare up to three domains, and select one. The sequence is shown in Figure 4-3 and the resulting screen in Figure 4-4.

LaunchName Project (devpost26)
Final System Requirements Specification for LaunchName

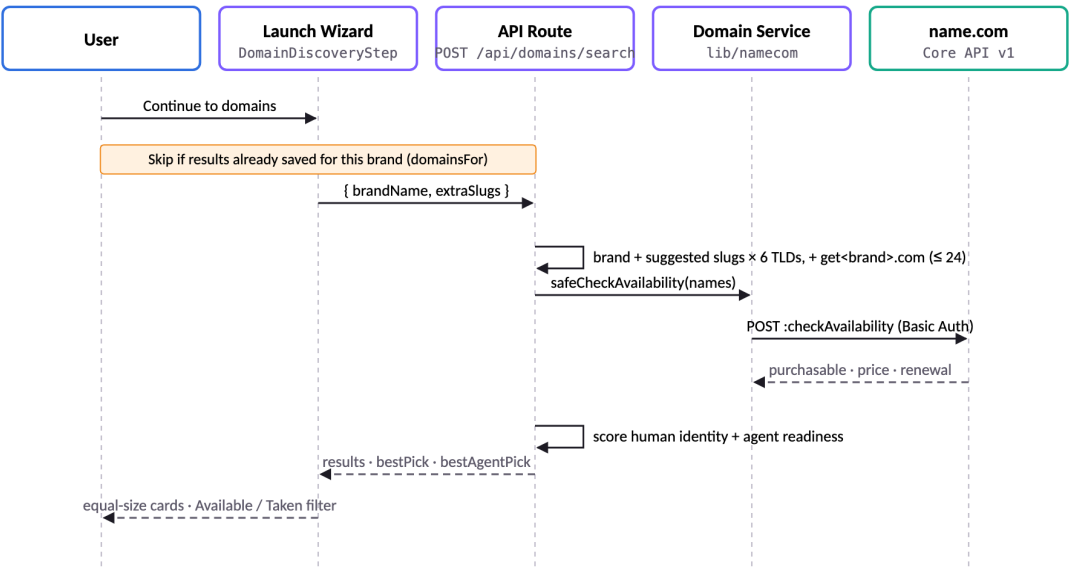


Figure 4-3: Domain discovery sequence

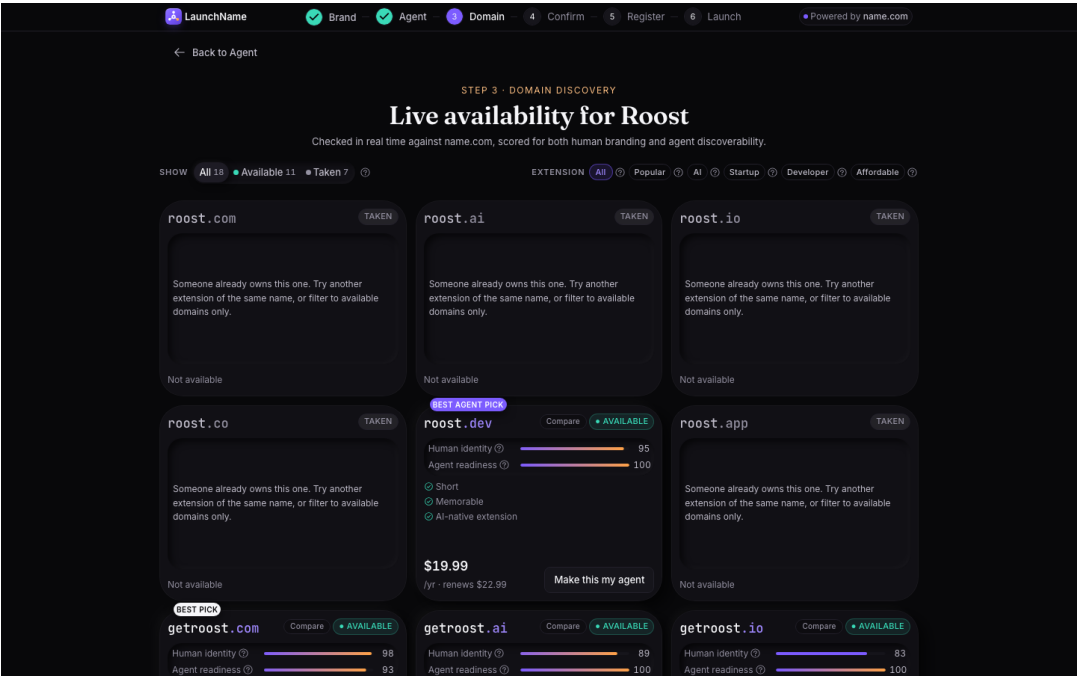


Figure 4-4: Domain discovery with equal-size cards, scores and filters

4.2.3 Functional Requirements

The functional requirements for the domain service are shown in Table 4-2.

Table 4-2: Domain Service functional requirements

Requirement ID	Description
REQ-IDS-1.1	The domain service shall build up to 24 candidates from the brand name and its suggested slugs across .com, .ai, .io, .co, .dev and .app.
REQ-IDS-1.2	The domain service shall check availability and price for every candidate through name.com, and shall show failed lookups as errors rather than dropping them.
REQ-IDS-1.3	The domain service shall score each available domain from 0 to 100 for human identity and for agent readiness, with the reasons behind each score.
REQ-IDS-1.4	The domain service shall mark a best pick for people and, when an agent is enabled, a best pick for agents.
REQ-IDS-1.5	The interface shall filter results by extension group and by availability (All, Available, Taken), showing a live count for each.
REQ-IDS-1.6	The interface shall display every result card at an identical size regardless of its content.
REQ-IDS-1.7	The domain service shall offer only the registration periods that the selected extension allows.
OPT-IDS-1.8	The domain service may substitute a deterministic mock provider when name.com credentials are absent.

4.3 Registration Orchestrator

The registration orchestrator turns a confirmed choice into a working launch. It purchases the domain, configures DNS and, when an agent is enabled, creates, publishes and verifies the agent. Each step reports its real result to the progress list; nothing is simulated.

4.3.1 Priority

The registration orchestrator is of high priority because it is the step that spends money and creates infrastructure.

4.3.2 Stimulus/Response Sequences

When the user selects Secure this domain, the orchestrator re-checks availability and, if the domain is still free, registers it with an idempotency key. It then creates the default DNS records, runs the agent steps if enabled, and completes the launch. If the domain was taken in the meantime, the user is offered another. Figure 4-5 shows the sequence and Figure 4-6 the Launch Center that follows.

LaunchName Project (devpost26)

Final System Requirements Specification for LaunchName

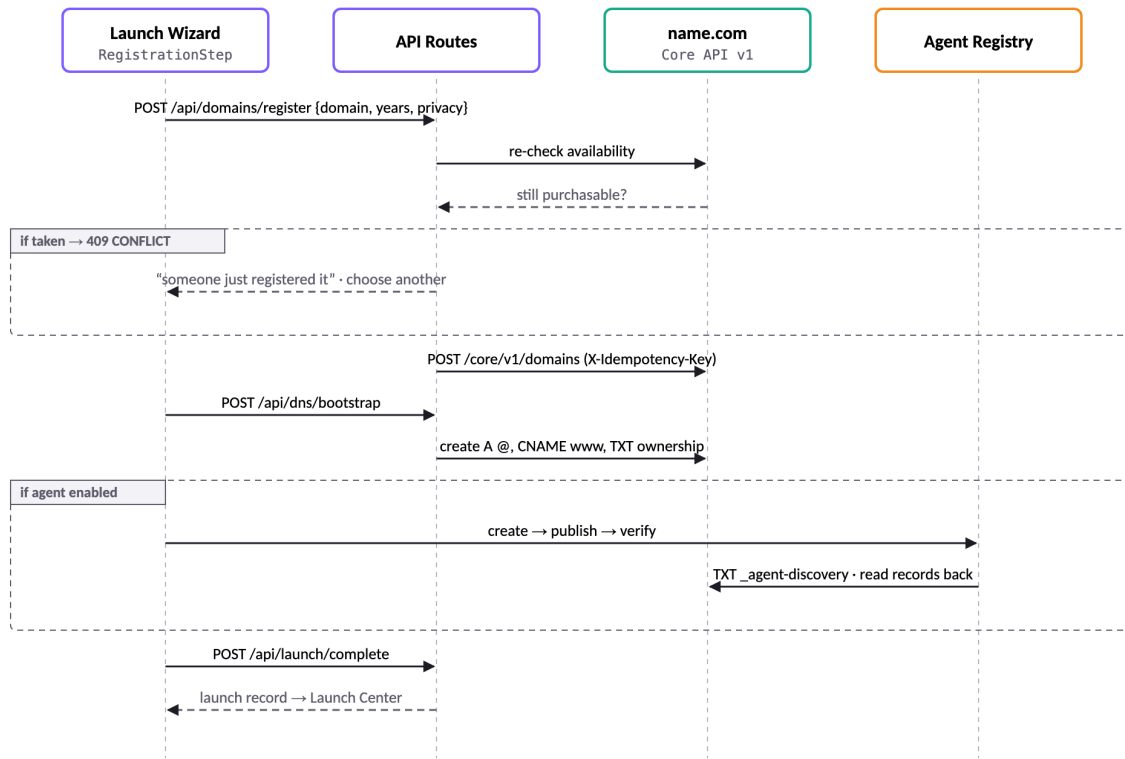


Figure 4-5: Registration and launch sequence

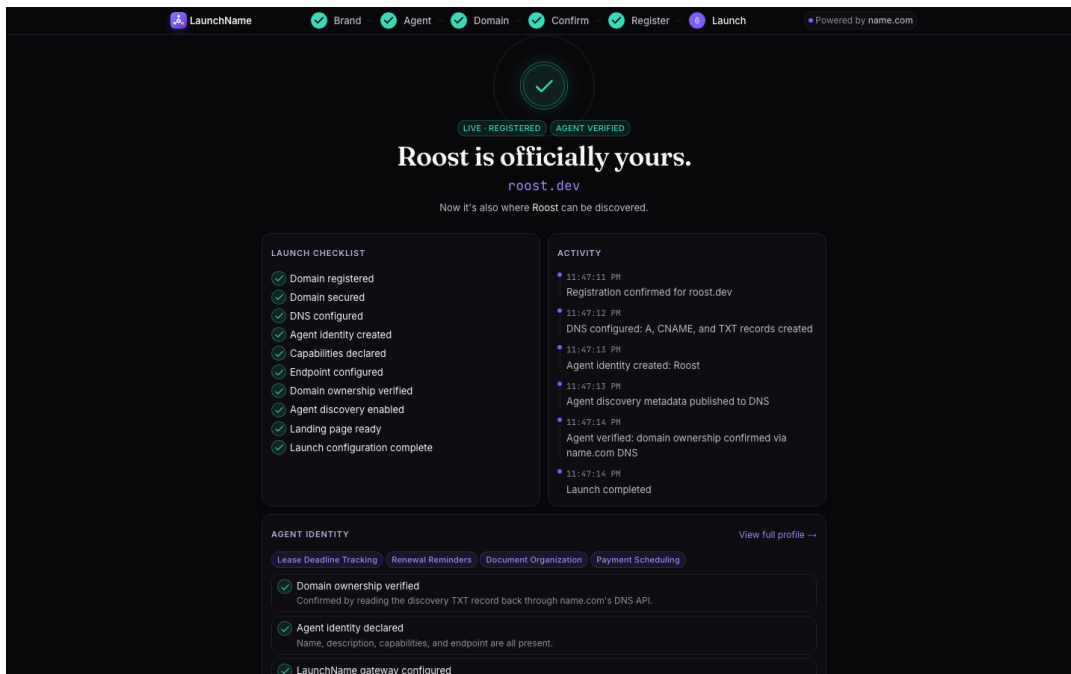


Figure 4-6: Launch Center after a successful registration

4.3.3 Functional Requirements

The functional requirements for the registration orchestrator are shown in Table 4-3.

Table 4-3: Registration Orchestrator functional requirements

Requirement ID	Description
REQ-IRG-1.1	The orchestrator shall re-check availability immediately before purchasing a domain.
REQ-IRG-1.2	The orchestrator shall return a CONFLICT error and offer to choose another domain when the domain is no longer available.
REQ-IRG-1.3	The orchestrator shall reject registration periods outside 1 to 10 years.
REQ-IRG-1.4	The orchestrator shall register the domain with WHOIS privacy when the user chooses it and the extension supports it; the privacy option shall be disabled when unsupported.
REQ-IRG-1.5	The orchestrator shall create an A record at the root, a www CNAME and an ownership TXT record after registration.
REQ-IRG-1.6	The orchestrator shall show progress for each step from actual API results and shall never display simulated progress.
REQ-IRG-1.7	When an agent is enabled, the orchestrator shall create, publish and verify it before completing the launch.

4.4 DNS Manager

The DNS manager lets operators read and change a domain's DNS records. It offers a plain-language Simple view and a raw Advanced view, validates every change on the server before it reaches name.com, and records each change in the domain's activity log.

4.4.1 Priority

The DNS manager is of high priority because a domain is only useful once its records point somewhere.

4.4.2 Stimulus/Response Sequences

When an operator adds or edits a record, the request is validated against the rules for its type. Invalid records are rejected with a message that explains how to fix them and are never sent to name.com. Valid records are written through the Core API, logged and shown immediately. Figure 4-7 shows the sequence and Figure 4-8 the Advanced view.

LaunchName Project (devpost26)
Final System Requirements Specification for LaunchName

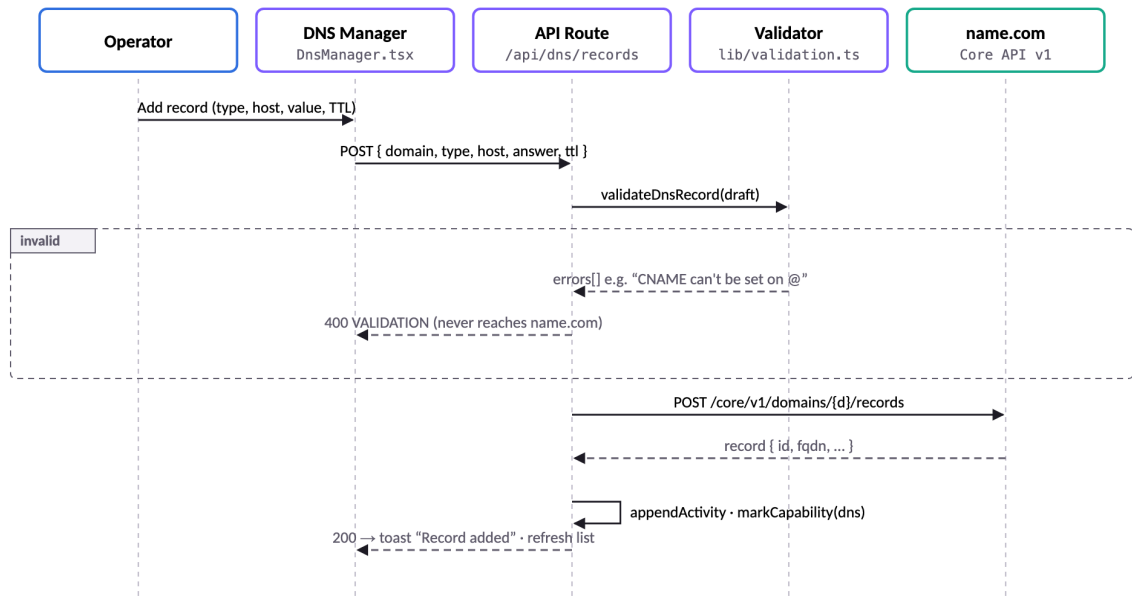


Figure 4-7: DNS record change sequence with server-side validation

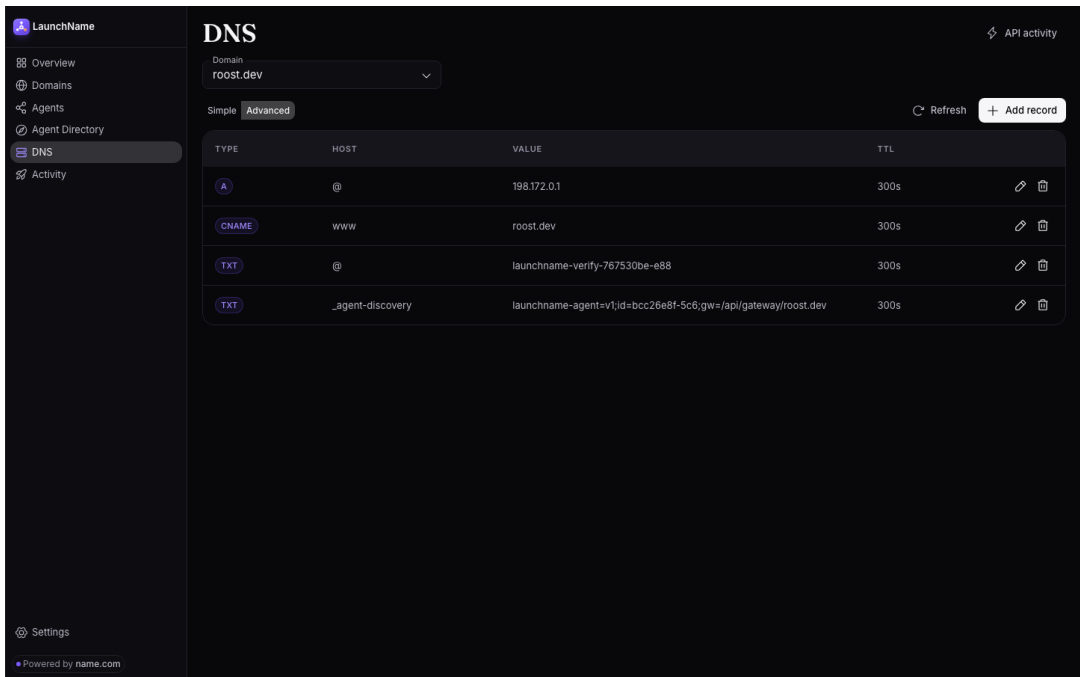


Figure 4-8: DNS manager, Advanced view

4.4.3 Functional Requirements

The functional requirements for the DNS manager are shown in Table 4-4.

Table 4-4: DNS Manager functional requirements

Requirement ID	Description
REQ-IDN-1.1	The DNS manager shall list all records for a domain (up to 500 per request).
REQ-IDN-1.2	The DNS manager shall create, update and delete A, AAAA, ANAME, CNAME, MX, NS, SRV and TXT records.
REQ-IDN-1.3	The DNS manager shall validate every record on the server before forwarding it to name.com.
REQ-IDN-1.3.1	TTL shall be between 300 and 86,400 seconds.
REQ-IDN-1.3.2	A and AAAA values shall be valid IPv4 and IPv6 addresses; CNAME, ANAME, NS and MX values shall be valid host names.
REQ-IDN-1.3.3	A CNAME record shall not be accepted at the root ("@").
REQ-IDN-1.3.4	MX and SRV records shall require a priority between 0 and 65,535; SRV values shall be "weight port target".
REQ-IDN-1.3.5	TXT values shall not exceed 2,048 characters.
REQ-IDN-1.4	The DNS manager shall accept underscore-prefixed hosts (such as _dmarc, selector._domainkey and _agent-discovery) and a wildcard host.
REQ-IDN-1.5	The DNS manager shall offer a Simple view with plain-language labels and an Advanced view with raw values.
REQ-IDN-1.6	The DNS manager shall label agent discovery records distinctly from other TXT records.
REQ-IDN-1.7	The DNS manager shall record every change in the domain's activity log.

4.5 Agent Registry

The agent registry stores each agent's identity, publishes a discovery marker to the domain's DNS, and verifies the agent. Its verifier runs four independent checks and reports exactly what each one found; no check implies another. The record format is LaunchName's own pointer, inspired by DNS-based agent discovery research (DNS-AID and ANS); it is not an official record type of either proposal and not a name.com feature.

4.5.1 Priority

The agent registry is of high priority because it is what lets a domain act as the address of an AI agent.

4.5.2 Stimulus/Response Sequences

When an agent is created, the registry stores its record. Publishing writes the discovery TXT record through name.com, adding an A record when the agent lives on a subdomain. Verification reads the domain's records back and requires an exact match, then probes the agent's endpoint and counts it as reachable only if the response identifies itself as that agent. Figure 4-9 shows the sequence and Figure 4-10 a verified agent's profile.

LaunchName Project (devpost26)
Final System Requirements Specification for LaunchName

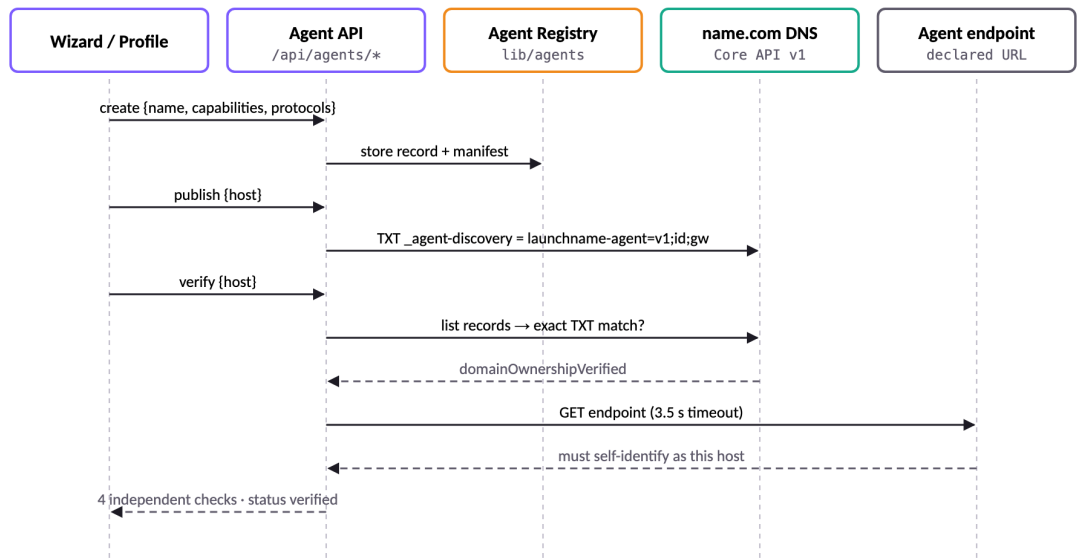


Figure 4-9: Agent publish and verify sequence

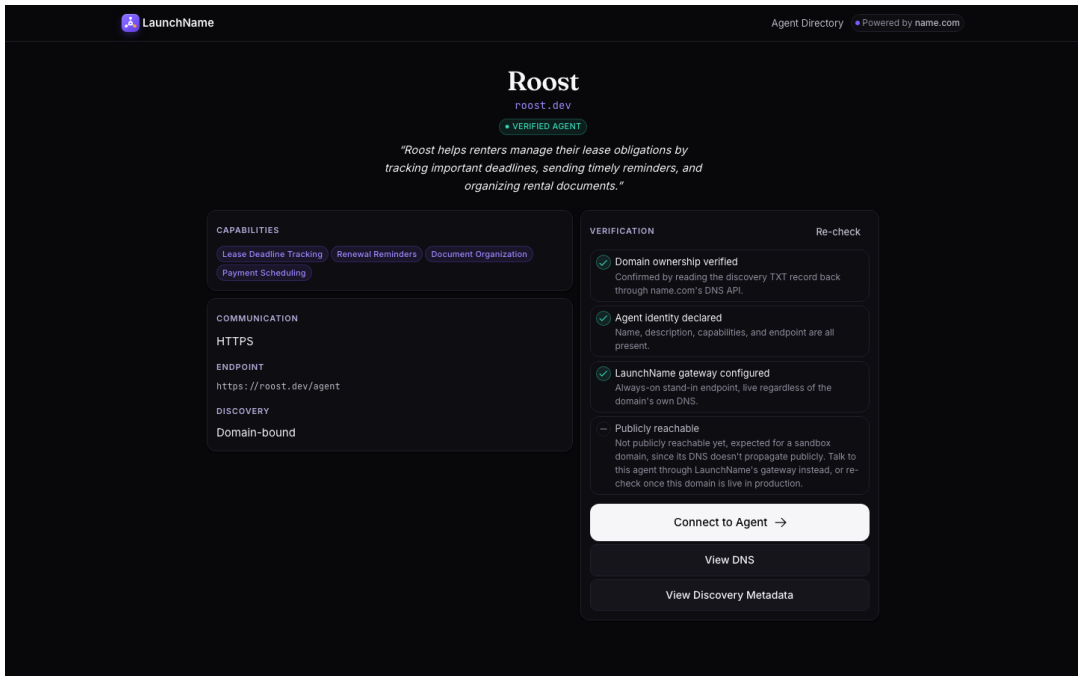


Figure 4-10: Agent profile showing the four verification checks

4.5.3 Functional Requirements

The functional requirements for the agent registry are shown in Table 4-5.

Table 4-5: Agent Registry functional requirements

Requirement ID	Description
REQ-IAG-1.1	The registry shall require a name, a description, at least one capability (at most eight) and at least one protocol (HTTPS, Streaming or MCP) to create an agent.
REQ-IAG-1.2	The registry shall publish a TXT record at _agent-discovery (or _agent-discovery.<subdomain>) with the value launchname-agent=v1;id=<12 characters>;gw=<gateway path>.
REQ-IAG-1.3	The registry shall create an A record for an agent published on a subdomain.
REQ-IAG-1.4	The verifier shall confirm domain ownership by reading the domain's records back through name.com and requiring an exact match of host and value.
REQ-IAG-1.5	The verifier shall report four independent results: domain ownership, identity declared, gateway configured and publicly reachable.
REQ-IAG-1.6	The verifier shall count an endpoint as publicly reachable only if it responds within 3.5 seconds and identifies itself as the agent's own host.
REQ-IAG-1.7	The registry shall let users find agents by capability, category, protocol and natural-language intent.
REQ-IAG-1.8	The interface shall never present the discovery record as an official DNS-AID or ANS record or as a name.com feature.

4.6 Agent Gateway

An agent gateway is an optional subsystem. If implemented, it lets one agent discover and message another through a visible handshake. When the target agent is publicly reachable, the gateway forwards the message to the agent's own endpoint; otherwise, as is always the case for sandbox domains, Claude answers in character as the agent, and the conversation is labeled as a demo.

4.6.1 Priority

The agent gateway is of medium priority. It is not required for registration but demonstrates what a discoverable agent makes possible.

4.6.2 Stimulus/Response Sequences

When a user sends a message, the gateway records each handshake step (discover, identify, verify, connect), chooses the live or demo connector, and returns the response with the full handshake log. Figure 4-11 shows the sequence and Figure 4-12 a completed exchange.

LaunchName Project (devpost26)

Final System Requirements Specification for LaunchName

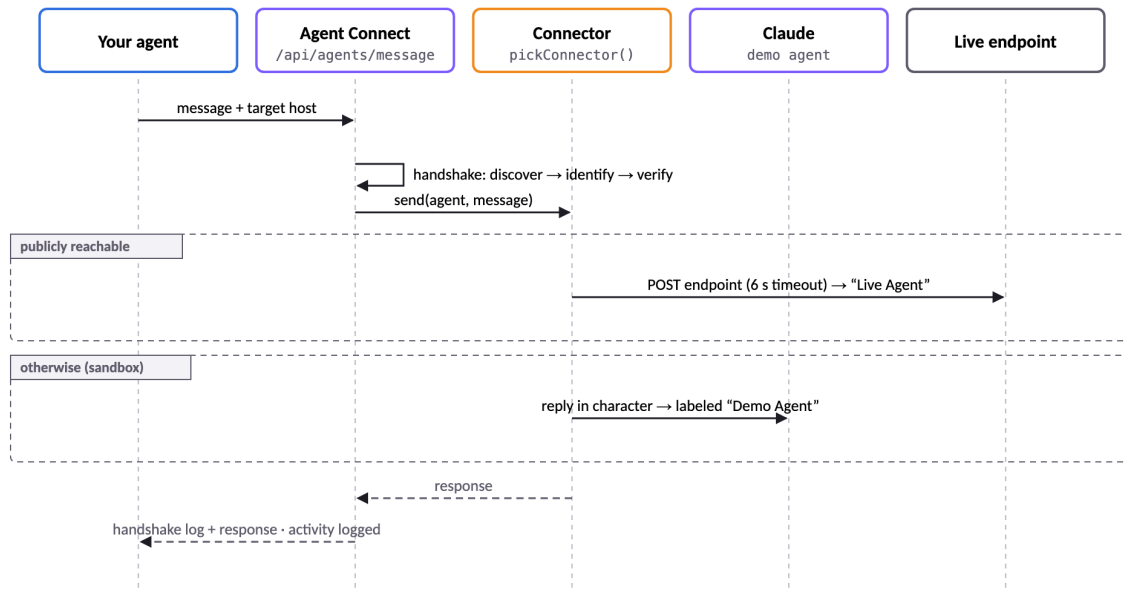


Figure 4-11: Agent-to-agent message sequence

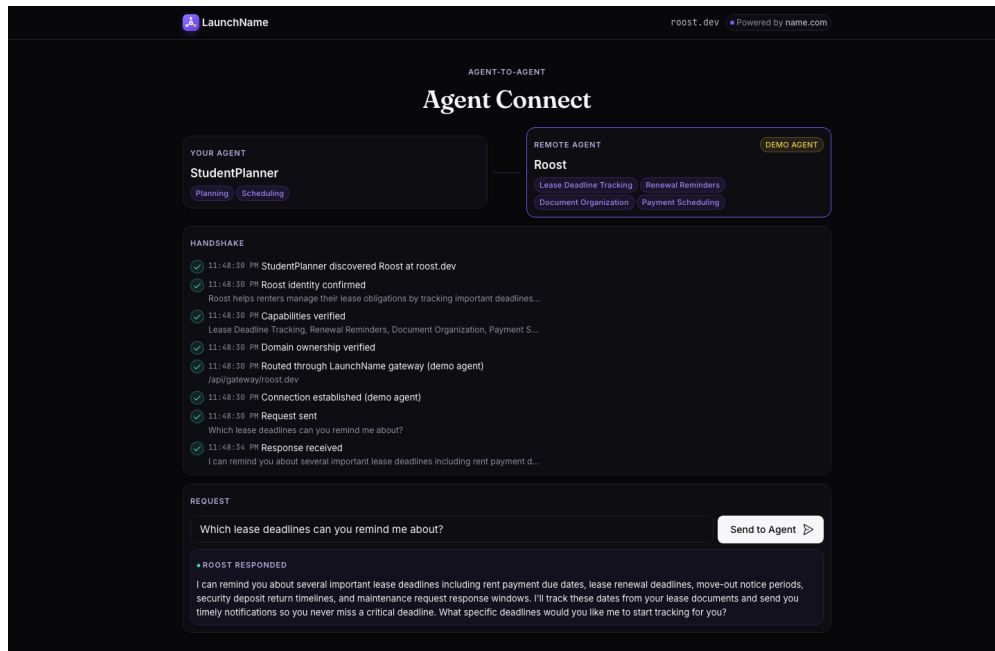


Figure 4-12: Agent Connect after a completed exchange

4.6.3 Functional Requirements

Since the agent gateway is an optional component, all requirements are flagged as optional recommended entries. When an agent gateway is present, the recommendations should be respected. The functional recommendations are shown in Table 4-6.

Table 4-6: Agent Gateway functional recommendations

Requirement ID	Description
OPT-IGW-1.1	If implemented, the gateway would show each handshake step: discover, identify, verify and connect.
OPT-IGW-1.2	If implemented, the gateway would forward messages to the agent's own endpoint, with a 6-second timeout, when the agent is publicly reachable.
OPT-IGW-1.3	If implemented, the gateway would otherwise generate an in-character reply with Claude and label the agent "Demo Agent".
OPT-IGW-1.4	If implemented, the gateway would log each exchange in the domain's activity stream.

5. Nonfunctional Requirements

This section describes requirements related to the performance, quality and security attributes of the platform that are separate from the functional requirements. Each subsection is separated by a set of nonfunctional requirements.

5.1 Performance Requirements

The performance requirements are capabilities the system needs to provide under normal operating conditions. Figure 5-1 shows response times measured on the development server against the live name.com sandbox and Claude API (three runs per endpoint). name.com operations complete in well under a second at the median, while AI generation takes several seconds and therefore relies on loading states rather than speed. Each performance requirement is listed in Table 5-1.

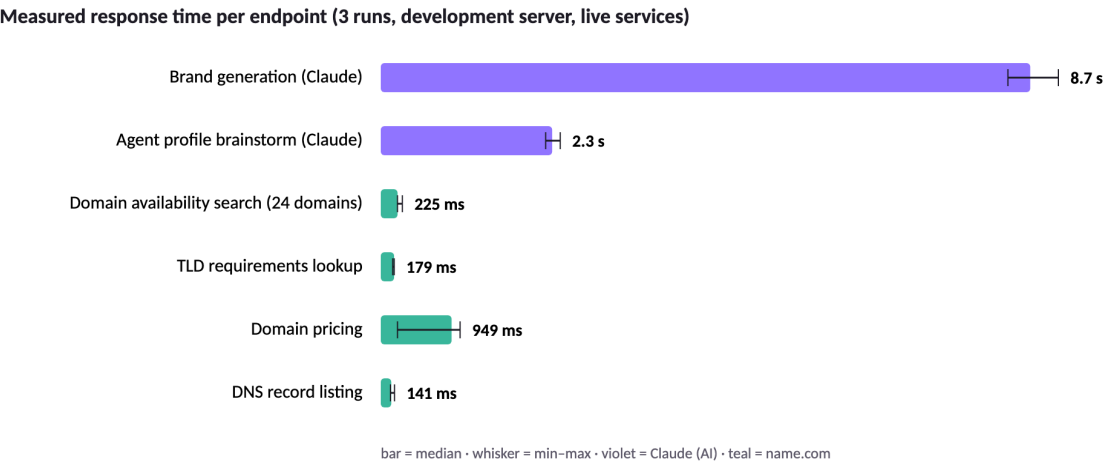


Figure 5-1: Measured response times per endpoint

Table 5-1: Performance requirements

Requirement ID	Description
REQ-PRF-1.1	Availability for a full candidate set (up to 24 domains) shall be returned within 1 second under normal conditions (measured median 225 ms).
REQ-PRF-1.2	Single name.com lookups (extension rules, pricing, DNS listing) shall complete within 2 seconds (measured medians 141 to 949 ms).
REQ-PRF-1.3	Every operation that can exceed 1 second shall show a contextual loading state, such as brand generation (measured median 8.7 s).
REQ-PRF-1.4	Interface animation shall use transform and opacity only, and all non-essential motion shall stop when the user prefers reduced motion.
REQ-PRF-1.5	All text shall meet WCAG AA contrast of at least 4.5:1 against its background.
OPT-PRF-1.6	The platform may store launches and agents in a database so that records survive serverless cold starts.

5.2 Security Requirements

Security requirements refer to the protection of credentials, user data and the integrity of what the platform reports. Credentials live only in server-side modules; the browser never sees them and never calls an external service directly. Communication with name.com and Anthropic uses TLS. Figure 5-2 shows the trust boundary. The security requirements are shown in Table 5-2.

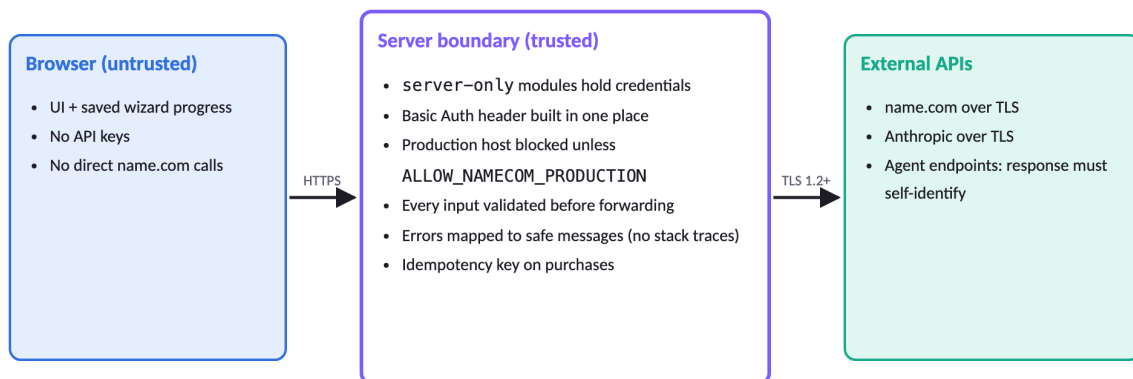


Figure 5-2: Security trust boundary

Table 5-2: Security requirements

Requirement ID	Description
REQ-SEC-1.1	API credentials shall be read only by server-side modules and shall never be sent to the browser.
REQ-SEC-1.2	Calls to name.com's production host shall be refused unless explicitly enabled.
REQ-SEC-1.3	All input shall be validated at the API boundary before it is forwarded to an external service.
REQ-SEC-1.4	Errors shall be mapped to safe, plain-language messages without internal details.
REQ-SEC-1.5	All communication with external services shall use TLS 1.2 or later.
REQ-SEC-1.6	An agent shall not be reported as reachable unless its endpoint identifies itself as that agent, so an unrelated site cannot produce a false verification.
OPT-SEC-1.7	The platform may require users to sign in (for example with Google) before public deployment, so that anonymous visitors cannot register domains on the operator's account.

Appendix A. User Interface Screens

This appendix shows the implemented interface. All screens were captured from the running application connected to the live name.com sandbox and Claude API.

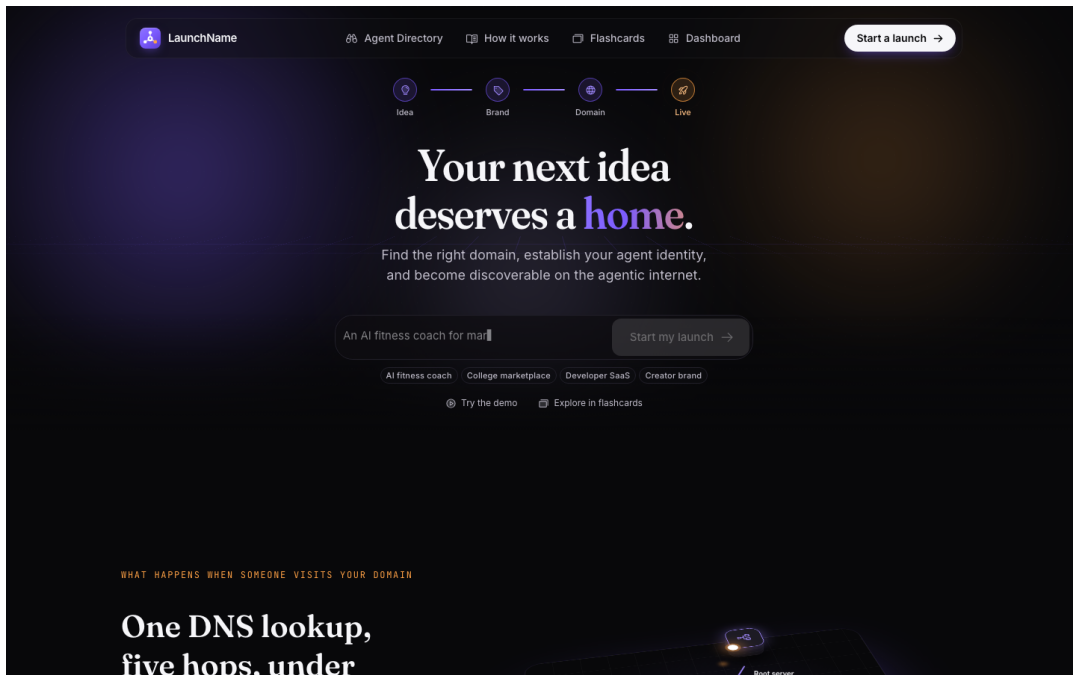


Figure A-1: Landing page with milestone tracker and animated idea field

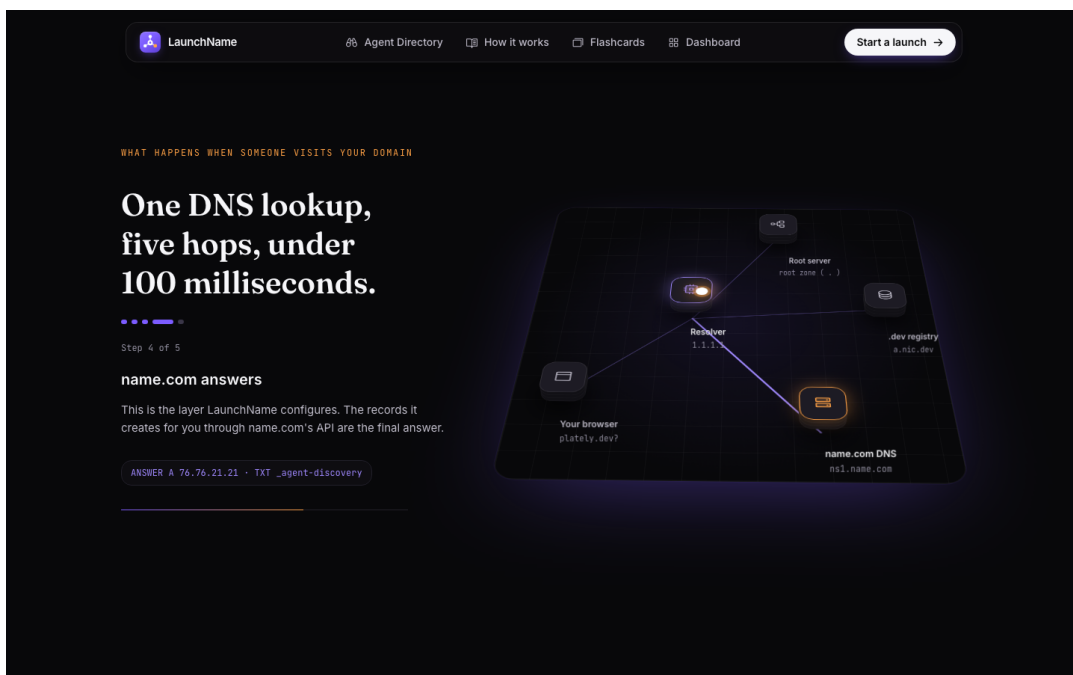


Figure A-2: Scroll-driven 3D DNS lookup on the landing page

LaunchName Project (devpost26)

Final System Requirements Specification for LaunchName

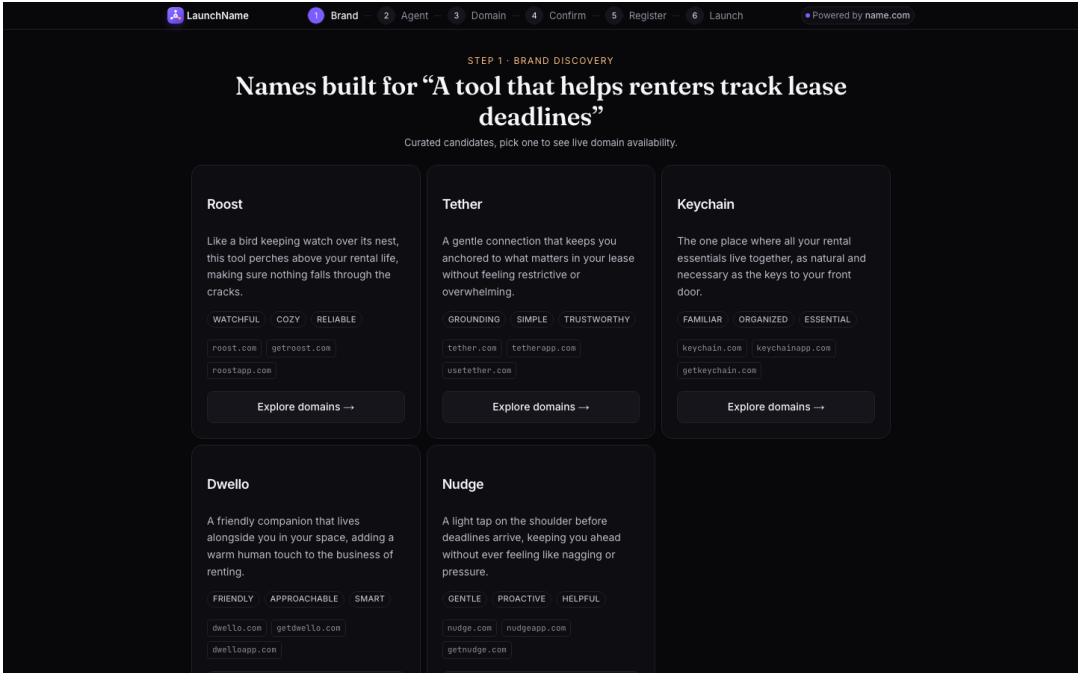


Figure A-3: Brand discovery step

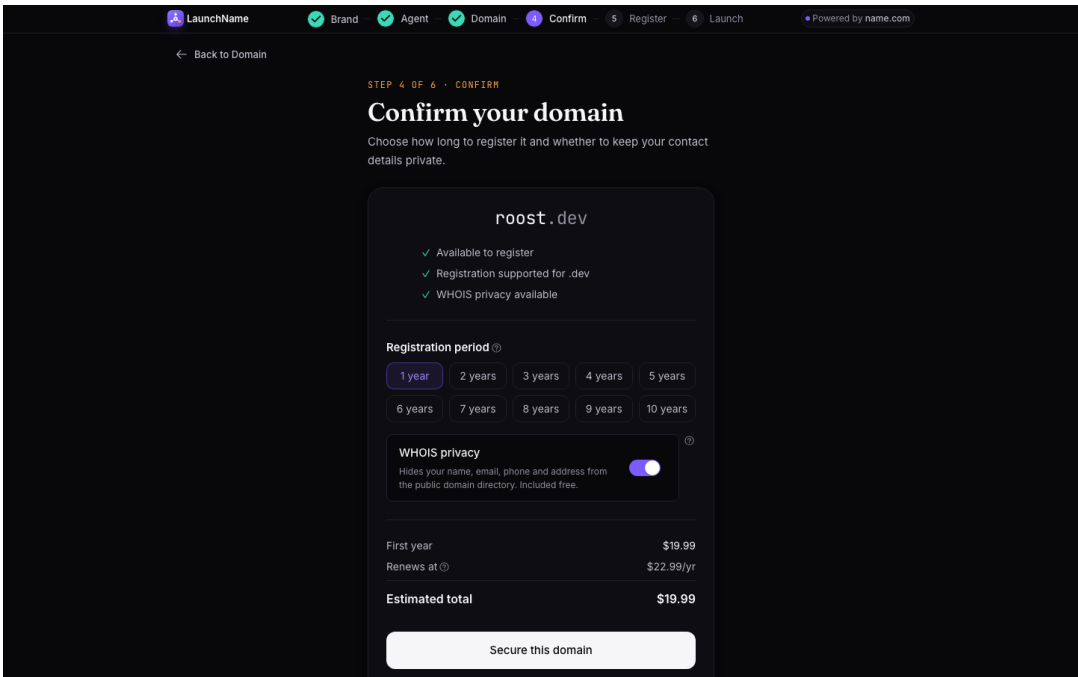


Figure A-4: Confirm step with registration period, WHOIS privacy and help tips

LaunchName Project (devpost26)

Final System Requirements Specification for LaunchName

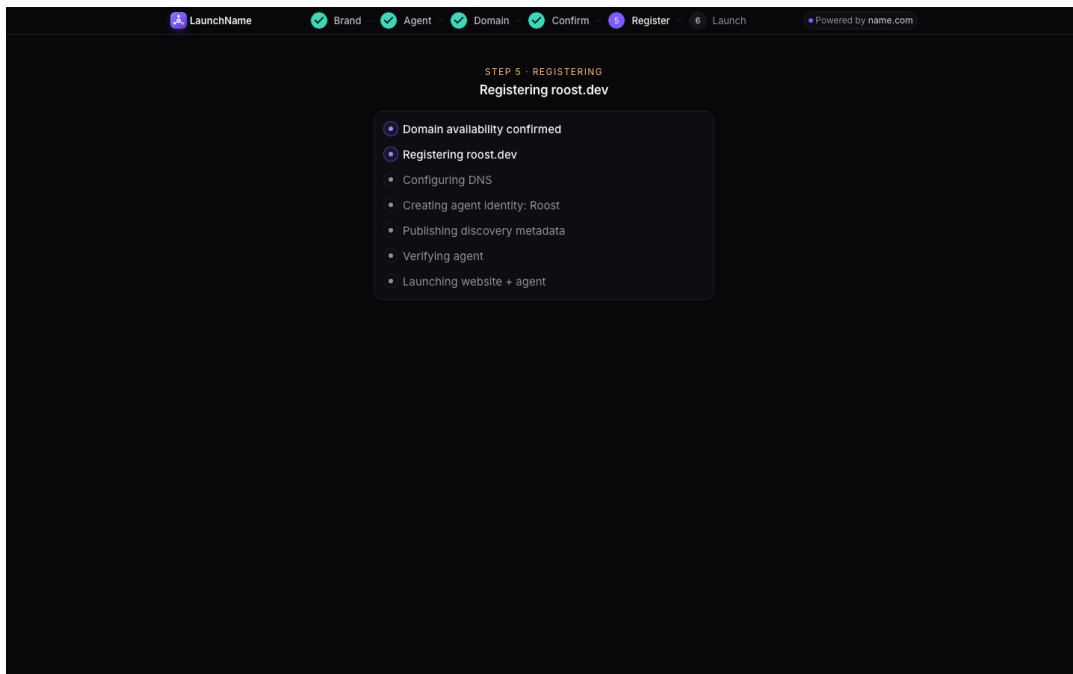


Figure A-5: Registration progress driven by real API results

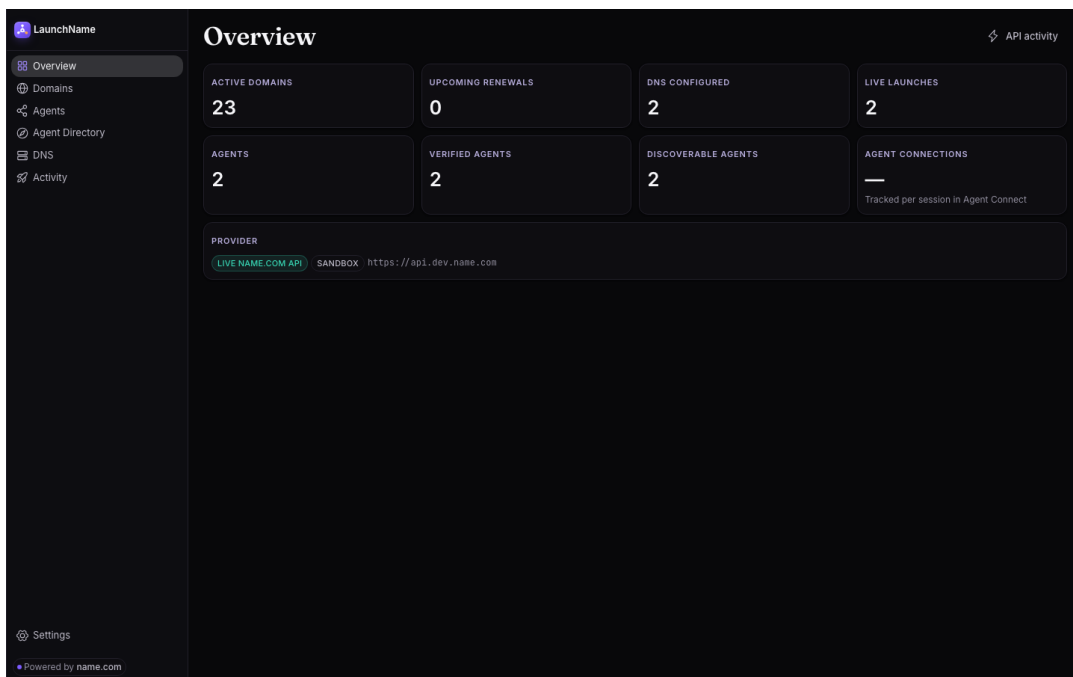


Figure A-6: Operator dashboard overview

LaunchName Project (devpost26)

Final System Requirements Specification for LaunchName

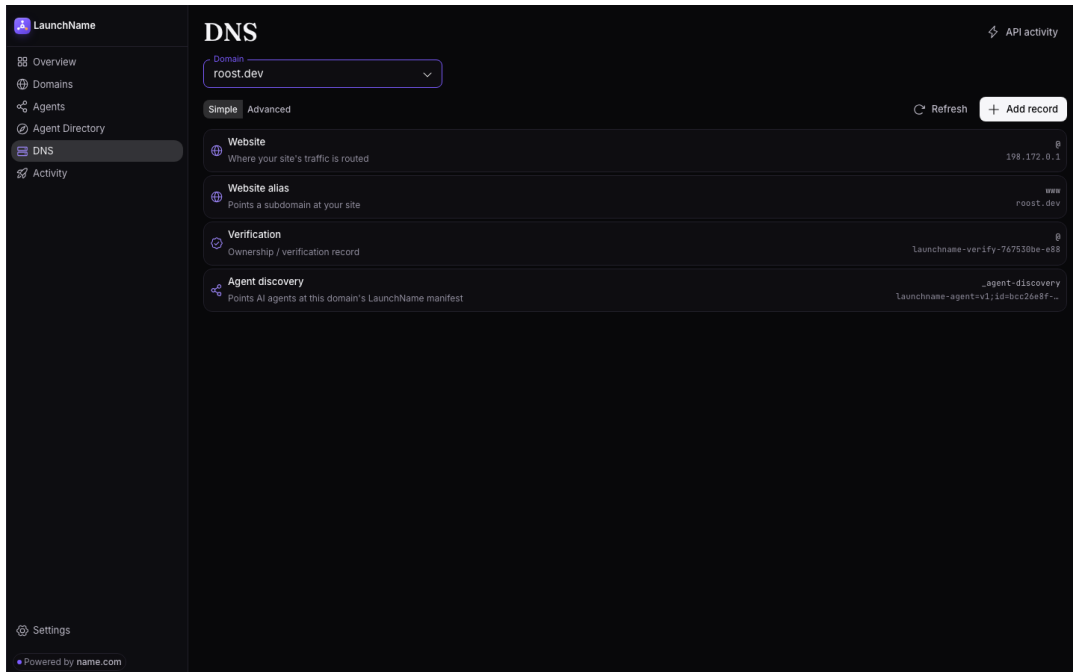


Figure A-7: DNS manager, Simple view, including the agent discovery record

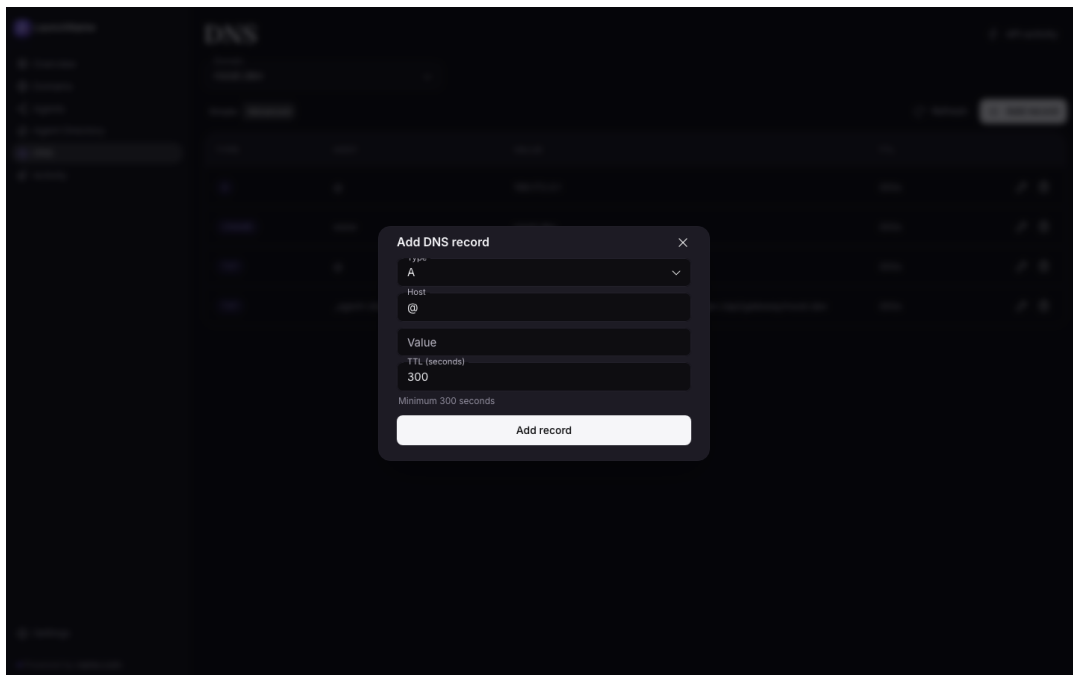


Figure A-8: Add DNS record dialog

LaunchName Project (devpost26)

Final System Requirements Specification for LaunchName

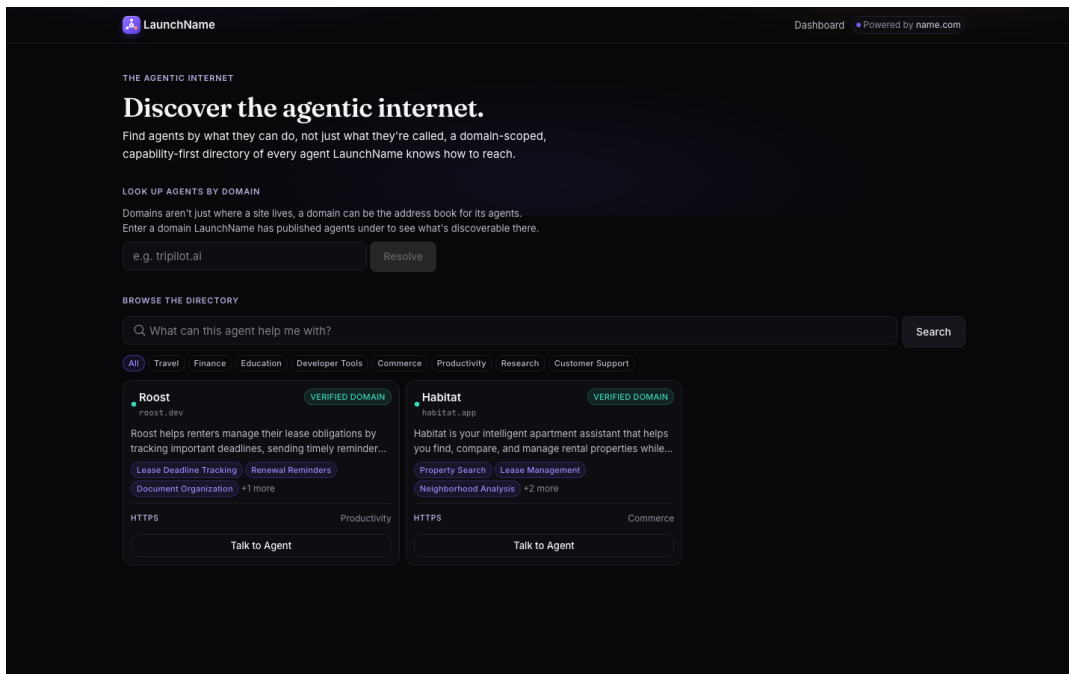


Figure A-9: Agent directory

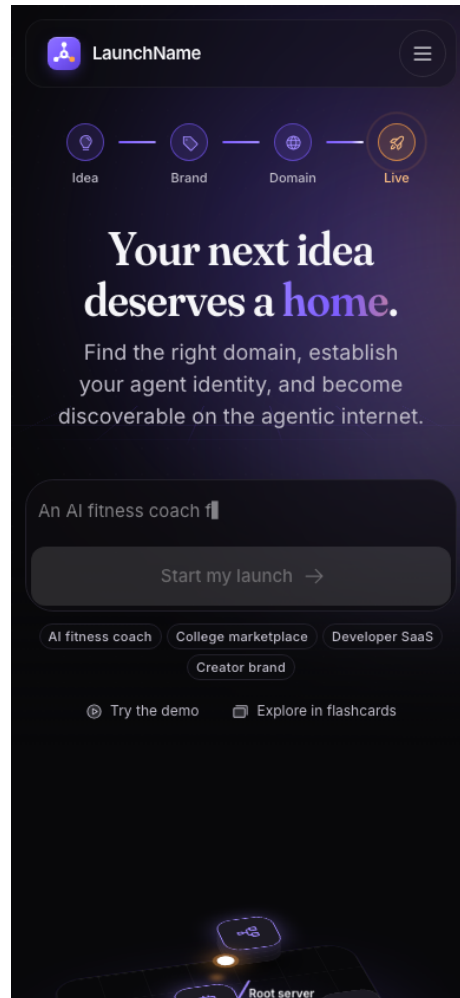


Figure A-10: Landing page on a 390-pixel-wide phone

Appendix B. Requirements Traceability Matrix

Table B-1 links each requirement to the code that implements it and the test or inspection that verifies it. File paths are relative to the launchname-prototype repository.

Table B-1: Requirements traceability matrix

ReqID	Implementation	Verification
REQ-EXT-1.1 to 1.6	lib/namecom/client.ts	Code inspection; integration suites
REQ-EXT-1.7	lib/namecom/domains.ts · /api/domains/tld-requirements	domains.test.ts (.ai minimum)
REQ-EXT-2.1, 2.2	lib/brand.ts · lib/agents/brainstorm.ts	E2E browser run; mock mode in tests
REQ-EXT-3.1 to 3.3	app/api/** · lib/http.ts	All integration suites
REQ-IWZ-1.1 to 1.8	context/LaunchContext.tsx · features/launch/LaunchWizard.tsx	launchWizard.test.ts (12) · 13-check E2E navigation run
REQ-IWZ-1.9	components/ui/HelpTip.tsx	E2E (tooltip opens on hover)
REQ-IDS-1.1 to 1.4	app/api/domains/search/route.ts · lib/agents/scoring.ts	domains.test.ts
REQ-IDS-1.5, 1.6	lib/domainFilters.ts · components/domain/DomainCard.tsx	domainFilters.test.ts · E2E (cards measured 262 × 331 px)
REQ-IDS-1.7	features/domains/DomainDecisionStep.tsx	domains.test.ts · E2E
REQ-IRG-1.1 to 1.3	app/api/domains/register/route.ts	domains.test.ts (conflict, period)
REQ-IRG-1.4 to 1.7	features/registration/RegistrationStep.tsx · /api/dns/bootstrap	dns.test.ts (bootstrap) · live E2E registration
REQ-IDN-1.1 to 1.7	lib/validation.ts · lib/namecom/dns.ts · /api/dns/records	validation.test.ts (12) · dns.test.ts (5)
REQ-IAG-1.1 to 1.8	lib/agents/* · /api/agents/*	agents.test.ts (5)
OPT-IGW-1.1 to 1.4	lib/agents/connector.ts · /api/agents/message	Live E2E agent exchange
REQ-PRF-1.1 to 1.3	API routes · loading states	Measured latency (Figure 5-1)
REQ-PRF-1.4, 1.5	theme/tokens.ts · theme/components.ts	palette.test.ts (24) · reduced-motion E2E
REQ-SEC-1.1 to 1.6	lib/namecom/client.ts · lib/agents/verification.ts	Code inspection · agents.test.ts

Appendix C. Verification Summary

The platform is verified by an automated suite of 71 tests (Table C-1) run with Vitest. Integration tests call the real API route handlers against the in-repository name.com mock, so they are deterministic and do not spend sandbox balance. Browser runs against the live sandbox complement them. The latest run is shown in Figure C-1.

Table C-1: Automated test suites

Suite	Type	Tests	Covers
launchWizard.test.ts	Unit	12	Navigation, saved progress, locking, stale responses
validation.test.ts	Unit	12	DNS record rules, underscore and wildcard hosts
domainFilters.test.ts	Unit	5	Availability and extension filters
palette.test.ts	Unit	24	Golden-ratio palette and WCAG AA contrast
domains.test.ts	Integration	8	Search, extension rules, registration, conflicts
dns.test.ts	Integration	5	DNS bootstrap, add/edit/delete, validation
agents.test.ts	Integration	5	Create, publish, verify, un-verify, directory
Total		71	All passing

```
$ npm test
✓ tests/unit/launchWizard.test.ts (12 tests)
✓ tests/unit/validation.test.ts (12 tests)
✓ tests/unit/domainFilters.test.ts (5 tests)
✓ tests/unit/palette.test.ts (24 tests)
✓ tests/integration/domains.test.ts (8 tests)
✓ tests/integration/dns.test.ts (5 tests)
✓ tests/integration/agents.test.ts (5 tests)
Test Files 7 passed (7)
Tests 71 passed (71)
```

Figure C-1: Latest test run